ChatGPT

分层持久记忆在线三维重建研究综述与落地方案评估

执行摘要

你们提出的“**短程 query-driven 工作记忆 + 活动在线状态 + 归档子图库（submap bank）+ 重访触发检索与恢复**”如果拿来对比 **LoGeR（arXiv:2603.03269）**，在**目标行为**与**机制侧重点**上确实是显著不同的：LoGeR 的核心是 chunk-to-chunk 连续传播（SWA 负责相邻 chunk 精对齐，TTT 快权重负责跨多 chunk 的长程压缩与尺度稳定）。[1]
但如果把对比范围扩大到 arXiv 上更广的相关工作，你们的“归档—检索—恢复（archive/retrieve/recover）”主张与 **VGGT-SLAM 2.0（arXiv:2601.19887）**、**CogniMap3D（arXiv:2601.08175）** 等**显式 submap + loop closure/revisit 检索 + 全局校正**路线存在明显重叠；因此“绝对意义上的首次提出”很难成立，创新空间更适合落在：**把 SLAM 式的检索/验证/恢复机制，与 TTT3R/LoGeR 这类长上下文 feed-forward/在线状态更新框架进行“机制级融合”**，并用针对性的 stress tests（branch ambiguity、long-gap revisit、budget-fixed horizon scaling、forced backtracking）证明“连续传播式 hybrid memory 不足以覆盖的失败模式”。[2]

下面给出你要求的：新颖性结论（含证据）、可落地的设计变体清单（带取舍与工程工作量）、对比表、评测协议与指标、推荐 baselines/ablations，以及未来扩展路线图（含两张 mermaid 图）。

新颖性评估与研究定位

结论式新颖性判断

(a) 简明新颖性评估（按你们最关心的两个参照系给出 Yes/No）

- **相对 LoGeR（arXiv:2603.03269）：Yes（新颖）**
  证据：LoGeR 明确采用“分 chunk 顺序处理 + 跨 chunk 的两条信息通路”：其跨 chunk 机制是 **(i) chunk-wise TTT（快权重 apply-then-update）** 与 **(ii) 仅邻接 chunk 的 SWA（lossless 信息通路）**，并且为了极长时程可选用“周期性 state reset”来抑制漂移；论文叙事与机制都围绕“连续流中的 chunk-to-chunk coherence 与尺度稳定”，并未把“显式归档 submap、远历史检索、重访触发恢复”作为主要组件展开。[1]
  这意味着：只要你们把贡献点写成“**persistent archived memory + revisit-triggered retrieval + recovery-based correction**（跨长时间间隔的再激活）”而不是“也有 short/long memory”，在 LoGeR 语境下是清晰差异化。[3]

- **相对更广泛 arXiv 相关工作：No（概念并非首创），但可主张“机制组合与评测主张的新颖”**
  关键证据链（说明“归档/检索/恢复”思想已在 arXiv 出现）：

- **VGGT-SLAM 2.0（arXiv:2601.19887）**：显式维护 submaps，并在“当前 submap ↔ 历史 submaps”之间做图像检索（SALAD），再利用 VGGT attention 层做检索候选的匹配验证，建立 loop closure 约束并进行全局优化以恢复全局一致性。[4]
- **CogniMap3D（arXiv:2601.08175）**：提出“持久 memory bank 的静态场景存储与快速检索”，并在重访时检索存储场景、重定位相机、用新观测更新记忆，同时结合 factor graph optimization 精炼位姿。[5]

- **SLAM3R（arXiv:2412.09401）**：两级体系（局部滑窗重建 + 全局注册），并维护“有限 reservoir 的 scene frames”，每次注册新 keyframe 时检索“最相关的 scene frames”作为参考；这是“受限持久集合 + 按需检索参考帧”的系统化实现。 [6]

因此，你们更稳的“可发表创新点”不应写成“我们首次做 submap 归档与重访检索”，而应写成更具体、可验证的机制主张，例如： 1) **在 LoGeR/TTT3R 这种“连续传播式长上下文重建”框架上，引入“事件触发的归档—检索—恢复生命周期”，且恢复仅在重访/歧义时触发（非持续后端）。** [7]
2) **在固定预算（O(1) GPU 状态/缓存）约束下，把历史信息从“单一状态被覆盖”改为“可检索 archive bank”，并证明这能显著降低 branch confusion 与 long-gap revisit 的失败率。**（预算约束与 cache/anchor 技术可借鉴 OVGGT、FrameVGGT 等） [8]

LoGeR vs 分层持久记忆 关键维度对比表

(c) 关键维度对比（面向你们写 related work / contribution）

维度	LoGeR（arXiv: 2603.03269）	你们的分层持久记忆（目标设计）
核心目标	连续长视频流中 chunk-to-chunk 一致性与全局尺度稳定（无 post-optimization 强调） [9]	长时间间隔后“找回历史区域、消歧分支、恢复全局身份一致性”，并在必要时校正漂移（事件触发）
短程机制	邻接 chunk 的 SWA：仅在相邻 chunk 边界建立 lossless 信息通路（少量层插入） [10]	query-driven working memory：维护 token/anchor、产出可检索摘要/descriptor、为 archive 与检索服务（你们需明确接口与预算）
长程机制	chunk-wise TTT 快权重（apply-then-update），压缩跨 chunk 全局上下文；可选周期性 reset 抑制极长漂移 [10]	active online state（可沿用 TTT3R 的置信引导更新）+ archived submap bank（CPU/磁盘可扩展），显式生命周期（observe→promote→archive→retrieve→recover）
Retrieval	未作为核心机制出现：主要是顺序传播与可选 reset [10]	需要：重访/歧义触发，从 archive bank 检索 top-k 候选 submap/token
验证	相邻 chunk 对齐主要依赖 SWA 保真通路 [10]	推荐：两段式验证（外观检索→几何/注意力验证），避免重复结构假阳性；可参考 VGGT-SLAM 2.0 的“检索 + attention 验证 + loop-closure edge”范式 [4]
恢复/校正	强调无后端；主要靠 TTT+SWA 维持一致；必要时 reset [10]	recovery-based correction：把检索到的历史作为“纠错证据”，做轻量对齐与状态注入（可选小规模 merge/BA）
计算/预算控制	线性按 chunk 推进；SWA 仅邻接；TTT 为固定大小快权重 [3]	GPU 侧需固定预算（working+active），archive 可放 CPU；可借鉴 OVGGT/FrameVGGT 的 cache/anchor/rolling bank 思路做驱逐与保护 [11]

技术设计选项与落地配方

设计总原则

你们要“看起来显著不同”，关键不是换零件，而是把**跨层接口**与**触发逻辑**讲清楚：
- **短程层输出什么可检索表征？**（descriptor / prototype / anchor set）
- **何时写入 archive？以什么粒度？**（chunk/submap/frame-block）
- **触发检索的信号是什么？**（置信度、歧义度、回环先验）
- **检索后如何验证与恢复？**（避免假阳性，控制预算）

这些点可以“直接建立在 TTT3R/LoGeR 的现成结构上”，因为它们本身已经提供：在线状态（TTT3R）与 chunk 切分 + overlap 的天然子图边界（LoGeR）。[12]

(b) 可实现设计变体清单（按优先级与可落地性排序）

> 工程工作量给一个粗尺度：**S（1–2 周）/ M（3–6 周）/ L（>6 周）**，假设已有 TTT3R/LoGeR 可运行环境。

变体 A：子图快照归档 + 检索触发“快权重回滚”恢复（推荐起步，S）
- 核心：把 **LoGeR 的 chunk-wise TTT fast weights** 或 **TTT3R 的 state tokens** 在 “submap 完结时”做快照，并为每个快照存一个 descriptor（见后文）。重访触发时，从 archive bank 取回候选快照，进行验证后**直接替换/部分替换**当前 fast weights/state。
- 依托来源：LoGeR 的 TTT 明确是“apply-then-update 的 fast weights”，并且已经存在“state reset”概念，因此“回滚到历史快照”是自然改动。[10]
- 优点：改动最小、闭环最快；可直接把“revisit recovery”做出来。
- 风险：全量替换可能导致“抹掉当前新证据”，需要再加一个“混合策略”（见变体 B）。

变体 B：MeMix 式局部保留/局部覆盖的恢复注入（M）
- 核心：借鉴 MeMix 将 state 切成多个 memory patches，只更新/覆盖一部分，从而减少灾难性遗忘与误覆盖；在 recovery 时只对“被判定为与重访相关的 patch”注入历史快照。[13]
- 优点：比全量回滚更稳，能把“恢复”做成局部纠错。
- 风险：工程复杂度与调参成本上升；需要定义 patch 划分与选择规则（alignment score 等）。

变体 C：双键检索（dual-key retrieval）+ 两段式验证（推荐，M）
- 核心：每个 archive entry 存两类 key：
1) **外观/语义 key**（图像 global descriptor，例如 SALAD 类思路在 VGGT-SLAM 2.0 用于 loop closure 候选提出）；
2) **几何 key**（来自重建 token/pointmap 的几何摘要）。
检索先用外观 key 快速召回 top-k，再用第二阶段（几何或 attention-based overlap score）验证，避免重复走廊/相似办公室隔间假阳性。VGGT-SLAM 2.0 明确展示了“外观检索会误报，但可用 VGGT attention 信息做验证并过滤”。[4]
- 优点：最能支撑你们“branch disambiguation”的主卖点。
- 风险：需要额外实现验证器与阈值策略；但它能直接变成 ablation 点与数据结果亮点。

变体 D：事件触发预算分配（budgeted event-trigger router，M）
- 核心：把触发信号做成“预算分配器”：只有当不确定性/歧义度上升时才提高恢复预算（top-k、验证深度、是否做 merge）。TTT3R 的关键思想之一就是用“对齐置信度”导出闭式学习率，平衡保留历史与吸收新观测；这同样可以扩展成事件触发强度。[14]
- 优点：非常贴合“在线部署”叙事：平时轻量，关键时刻纠错。
- 风险：需要把多种信号（置信度、检索得分、几何残差）统一到可解释的 gating 规则。

变体 E：可选 MERG3R 式轻量 merge（只在检索命中时触发，L）
- 核心：重访命中且验证通过后，把“当前子图 + 历史子图”当作两个局部重建，做一次 MERG3R 风格的全局对齐 + 置信加权 BA 合并，但把规模限制在检索命中的少量子图上。MERG3R 明确是“分而治之：重排/划分/重建/合并”，合并阶段包含全局对齐与置信加权 BA。[15]
- 优点：恢复效果最强，最像“真正把地图接回去”。
- 风险：引入优化步骤后，要强调“事件触发 + 小规模”，避免被认为回到重型后端。

变体 F：Gaussian 子图实例化与融合（未来方向，L）
- 核心：将 archived submap 由 pointmap/token 升级为高斯子图或全局 gaussian map，重访时做子图对齐与融合，并使用渲染/几何一致性作为校正信号。SING3R-SLAM 明确采用“先构建局部一致 submaps，再逐步对齐融合到全局 Gaussian map，并用全局 map 反馈纠正 local drift”。[16]
- 优点：对 demo/系统价值极大（可视化、可渲染、可增量维护）。
- 风险：工程与算力成本高，且会引入更多优化与地图维护细节（与“纯重建”边界要写清）。

基于 TTT3R/LoGeR 的具体实现配方（可直接开工的“接口级”建议）

在 LoGeR（chunk-wise）上加 archive/retrieve/recover

LoGeR 已经具备天然 submap 粒度：**chunk + overlap**。[1]
可按以下 recipe 最小改动实现：

1) **Archive entry 定义（每个 chunk/submap 一条）**
- key：`desc_app`（外观）、`desc_geo`（几何）；
- value：`fast_weights_snapshot`（TTT fast weights）、`pose_chunk`、`pointmap_chunk`（可选压缩存 CPU）、`keyframes`（少量）。
LoGeR 的 chunk-wise TTT 结构明确有 fast weights，并且可以 reset，说明快照/恢复是语义一致的改动。[10]

2) **Descriptor 生成（两类，建议都做）**
- 外观：从每个 chunk 选 1–2 个代表帧，取 backbone/encoder 的 pooled embedding（实现上不依赖特定论文细节；如果要贴近 arXiv 证据，可复用 VGGT-SLAM 2.0 的“先检索后验证”范式）。[4]
- 几何：从 chunk 的 pointmap/pose 计算粗几何摘要（例如视角分布、尺度统计、稀疏关键点的全局池化），或直接池化“几何 token”。

3) **Trigger 信号**
- LoGeR 自身没有“置信度学习率”接口，但你可以用：
- chunk 内 pose/pointmap 的一致性残差（内部可定义），或
- 与上一 chunk overlap 匹配质量下降，或
- 外观检索命中分数高且时间间隔大。
同时保留 LoGeR 原有的“周期性 reset”作为对照 ablation。[10]

4) **Recovery 动作（从轻到重三档）**
- R1（最轻）：仅用检索到的历史 submap 作为“额外 anchor”，对当前 chunk 的全局相似变换进行一次校正（可仿照 Pi3-Chunk 的 overlap stitching 思路，但把 overlap 从“相邻 chunk”扩展到“检索命中 chunk”。[17]）
- R2：加载历史 fast weights 快照，重新对当前 chunk 执行一次 apply（必要时少量 update），实现“回到旧坐标系再继续”。[10]
- R3（可选）：加入小规模 MERG3R 式 merge（变体 E）。[18]

在 TTT3R（在线 state update）上加层级记忆

TTT3R 明确把重建视作 test-time training 的在线学习，并用“memory state 与观测对齐置信度”导出闭式学习率来平衡保留历史与吸收新信息。[14]
因此它更适合做“事件触发强度”与“state 注入”：

1) **Working memory（GPU 上固定预算）**
- 维护最近 `W` 帧的 lightweight tokens/anchors（类似 StreamVGGT 的 KV cache“隐式记忆”，但你们要做成“可查询、可摘要”。[19]）
- 如果担心 KV 增长，可直接借鉴 OVGGT/FrameVGGT 的“固定预算 bank + anchor 保护”的思想：
- OVGGT 用 Self-Selective Caching 与 Dynamic Anchor Protection 控制缓存与关键 token 的保留；[20]
- FrameVGGT 用“以帧为证据块”的 mid-term bank + 可选 anchor tier。[21]

2) **Archived bank（CPU/磁盘，可大）**
- 条目粒度：每 `K` 帧/每个局部轨迹段/每个运动子图；
- value：`state_snapshot`（或其压缩原型）、少量关键帧索引、子图 pose/pointmap 摘要。

3) **Trigger 与注入**
- 触发信号可以直接复用/扩展 TTT3R 的对齐置信度：当置信度下降（“对得不好”）时触发检索/恢复，并将置信度映射为：
- 检索 top-k 大小；
- 是否执行几何验证；
- 注入强度（替换 vs 混合）。[22]
- 注入策略可参考 MeMix“只改最不对齐的 patch、保留其他”的原则，以减少错误覆盖。[23]

评测协议与 Stress Tests 设计

(d) 指标体系：必须把“新颖性主张”量化

你们需要同时报告三类指标，否则很难证明“检索与恢复”而非“更强连续建模”带来的提升。

重建与位姿主指标（与 LoGeR/TTT3R/OVGGT 等对齐） - 位姿：ATE（以及需要时 RPE）——LoGeR 在 KITTI/ScanNet/TUM-Dynamics 等使用 ATE，并评估到 1k 帧/数千帧。[24]
- 重建：Accuracy / Completeness / Normal Consistency（7-Scenes、NRGBD 等在 OVGGT、CogniMap3D 中使用）。[25]

检索与验证指标（直接对应你们“revisit-triggered retrieval”） - Recall@k / Precision@k：重访事件下检索是否命中正确历史 submap；
- False loop closure rate：重复结构下误检率（VGGT-SLAM 2.0 证明“外观检索会误报，需验证过滤”——你们可复现此类现象并量化）。[4]
- Verification acceptance rate：候选经验证被接受的比例（配合阈值扫描曲线）。

恢复质量指标（对应“recovery-based correction”） - ΔATE：触发恢复前后 ATE 的瞬时下降量与稳态下降量；
- Drift slope：ATE 随时间/里程增长的斜率变化；
- Identity consistency：同一地点多次访问后的“同一坐标系一致性”（可用 loop closure 后误差回落与子图对齐残差衡量；VGGT-SLAM 2.0 明确有 loop closure + 全局优化后恢复全局重建的步骤）。[4]

(d) 实验协议与数据集选择（尽量复用 arXiv 已用设置）

为“说服力”，建议主线实验完全复用 LoGeR 与相关工作已采用的数据与长度设置，再叠加你们的 stress tests。

Horizon scaling（长程退化曲线）

- 数据：KITTI（最长 4661 帧、最高 5km）、VBR（LoGeR 重新适配用于更长更复杂轨迹）。 [26]
- LoGeR设置参考：KITTI 用 chunk size 32，VBR 用 chunk size 48；短序列评测常用 chunk size 64、overlap 3。 [27]
- 协议步骤：
  1) 对每条序列取前 N 帧（N ∈ {200, 500, 1000, 2000, 4000, full}）；
  2) 跑 LoGeR/TTT3R 及你们方法；
  3) 报告 ATE(N)、drift slope、显存/吞吐；
  4) 额外报告：你们方法在固定 GPU 预算下 archive 命中率与恢复次数（证明“事件触发”而非持续优化）。

Revisit recovery（重访恢复）

- 选择含回环的序列：LoGeR 在 KITTI 表中标注多条序列“Contains Loop”。 [10]
- 协议步骤（推荐事件级评估）：
  1) 将“重访事件”定义为：当前帧与历史某帧外观检索得分超过阈值且时间间隔 > T（T 自定）；
  2) 在每个事件点，记录触发前 ATE 局部窗口；
  3) 触发检索→验证→恢复；
  4) 记录恢复后同窗口 ATE 与子图对齐残差；
  5) 汇总 ΔATE、误触发率、恢复耗时。
- 对照：VGGT-SLAM 2.0 的 loop closure 流水线（检索 SALAD → attention 验证 → 加约束 → 全局优化）可作为“强但带后端”的参照上限/结构参照。 [4]

Branch disambiguation（分支消歧/重复结构）

这是你们最该“单列成主结果”的 stress test，因为它能证明“连续桥接”不足以处理“相似但不相同”的长期混淆。

- 数据候选：室内数据（ScanNet、7-Scenes、TUM-Dynamics）在 LoGeR、OVGGT、CogniMap3D 中均被使用。 [28]
- 协议建议（可控、可复现）：
  1) 从同一数据集中挑选两段外观相似但空间不同的走廊/房间片段 A 与 B（人工或基于检索相似度筛选）；
  2) 构造序列：…→A→长间隔过渡→B→…，并保证 A 与 B 在时间上相隔足够长；
  3) 评估：检索是否把 B 错连到 A；若错连，恢复是否能纠正（几何验证应拒绝假阳性）；
  4) 报告：false retrieval rate、verification rejection accuracy、branch confusion rate。
- 参考证据：VGGT-SLAM 2.0 明确展示“外观检索会把相似隔间误判为匹配，但其基于 attention 的 match score 能拒绝假阳性”，这正是你们想系统化解决的问题。 [4]

Forced backtracking（强制回退/回走）

该测试不一定在现有论文中作为标准协议出现，因此这里属于**你们提出的新 stress test**（来源未明确规定，需在论文中说明是你们设计的协议）。
- 协议：把序列切片为 forward 段 + reverse 段（或 forward 段 + 近似回到旧区域的绕行段），要求系统在回退过程中触发重访检索并恢复身份一致性。
- 指标：回退进入旧区域后，ATE 的恢复速度（ΔATE per frame）、恢复触发的时延（frames）、误恢复率。

- 为什么有效：它把“连续前进的稳定性”与“重访纠错能力”强行解耦——LoGeR 在连续前进上强，但这种测试更直接考察“远历史再激活”。 [10]

推荐 Baselines 与 Ablations

(e) 推荐 baselines（按对照价值分组）

连续传播式长上下文重建（你们要对标的“同赛道”） - LoGeR（arXiv:2603.03269）：SWA + chunk-wise TTT 的 hybrid memory，主打长视频 chunk coherence。 [3]
- TTT3R（arXiv:2509.26645）：对齐置信度导出闭式学习率的 training-free 在线状态更新。 [14]

流式/因果几何模型与预算控制（支撑你们“固定预算”主张） - StreamVGGT（arXiv:2507.11539）：因果注意力 + KV cache 作为隐式记忆。 [19]
- Evict3R（arXiv:2509.17650）：训练无关 token eviction，固定预算下保持性能。 [29]
- InfiniteVGGT（arXiv:2601.02281）：滚动记忆（rolling memory）与预算分配，主打无限流。 [30]
- OVGGT（arXiv:2603.05959）：Self-Selective Caching + Dynamic Anchor Protection，常量成本 streaming。 [31]
- FrameVGGT（arXiv:2603.07690）：以“帧证据块”为单位的 mid-term bank + anchor tier。 [32]
- MeMix（arXiv:2603.15330）：state 分块、选择性更新，缓解遗忘。 [33]

显式 submap / 检索 / 校正系统（你们必须正面对比，否则 novelty 站不住） - VGGT-SLAM 2.0（arXiv:2601.19887）：submap 间检索（SALAD）+ attention 验证 + loop closure + 全局优化恢复。 [4]
- CogniMap3D（arXiv:2601.08175）：持久 memory bank，多次访问检索与更新，factor graph 优化。 [34]
- SLAM3R（arXiv:2412.09401）：有限 scene frames reservoir + 检索最佳相关参考帧做注册。 [6]

可选的“合并式”上限参照 - MERG3R（arXiv:2603.02351）：分而治之重建 + 全局对齐 + 置信加权 BA 合并（更像离线/大规模），可用作“触发式 merge”的结构参考。 [15]

(e) 关键 ablations（必须做，且应与主张一一对应）

1) **无 archive bank**（仅 short+active，无可检索长期记忆）：验证“归档”必要性。
2) **有 archive 但无 retrieval**（只存不取）：验证“检索”必要性。
3) **有 retrieval 但无验证**：展示重复结构下假阳性灾难，证明“verification”必要性（可复刻 VGGT-SLAM 2.0 案例）。 [4]
4) **有检索+验证但无 recovery（只报告匹配，不注入/校正）**：证明“恢复校正”是性能来源。
5) **事件触发 vs 周期性触发**：对照 LoGeR 的 periodic reset 思路，证明“重访触发更省预算且更有效”。 [10]
6) **银行大小/预算扫描**：固定 GPU 预算下，扫描 archive 条目数与 descriptor 维度，画性能—成本曲线；并对照 OVGGT/FrameVGGT 的预算设定与随长度退化趋势。 [11]

机制图示

(f) 记忆生命周期流程图（mermaid）

```
flowchart TD
  A[Observe: incoming frames/chunk] --> B[Update active online state]
  B --> C[Update working memory\n(query-driven tokens/anchors)]
  C --> D{Archive trigger?\n(time/coverage/novelty)}
  D -- yes --> E[Summarize submap\n(prototype + descriptors)]
  E --> F[Store into archived bank\n(CPU/disk)]
```

```
  D -- no --> G[Continue streaming]

  H{Revisit/ambiguity trigger?\n(confidence drop / high similarity)} --> I[Retrieve top-k
candidates]
  I --> J[Verification\n(appearance + attention/geometry)]
  J -- pass --> K[Recovery-based correction\n(state injection / pose-map correction)]
  J -- fail --> G
  K --> B
```

(f) 重访触发恢复的时序图（mermaid）

```
sequenceDiagram
  participant S as Stream (frames/chunks)
  participant WM as WorkingMemory (GPU)
  participant AS as ActiveState (TTT fast weights/state tokens)
  participant AB as ArchivedBank (CPU/disk)
  participant V as Verifier (attn/geometry)
  participant R as Recovery (corrector)

  S->>AS: normal apply/update
  S->>WM: maintain query-driven tokens
  S->>AB: (optional) archive submap snapshot + descriptors

  Note over S,AS: long temporal gap / branch ambiguity
  S->>WM: compute query descriptor
  WM->>AB: ANN search top-k
  AB-->>WM: candidate submaps
  WM->>V: verify overlap/consistency
  V-->>WM: pass/fail + transform/score
  alt pass
    WM->>R: trigger recovery with best candidate
    R->>AS: inject/rollback/blend state
    R->>S: corrected poses/map alignment
  else fail
    WM->>AS: continue without correction
  end
```

可扩展方向与未来研究路线图

(g) 未来工作优先级清单（面向“能发论文/能做系统”的增量）

- **高优先：descriptor 设计与 dual-key 检索体系**
  直接对齐你们的 branch disambiguation 主张：采用“外观 key 快召回 + attention/几何验证强过滤”的两段式架构。VGGT-SLAM 2.0 已证明外观检索（SALAD）会误报，而用 VGGT attention 信息可验证并拒绝假阳性；你们可以把它抽象为通用模块并系统评测。 [4]

- **高优先：几何验证（geometric verification）轻量化**
  把验证从“重型优化”做成“可解释阈值 + 小规模对齐残差”，并形成 ROC/PR 曲线与误报分析；对照 VGGT-SLAM 2.0 的 match score/阈值设置，建立你们自己的验证器设计空间。 [4]

- **高优先：生命周期/驱逐策略（eviction & promotion）与固定预算**
  将“observe→promote→archive→retrieve→recover”具体化为可实现的策略，并在固定 GPU 预算下证明优势；可借鉴：

- OVGGT 的 Dynamic Anchor Protection（保护坐标关键 token）与预算点设置； [31]
- FrameVGGT 的“帧证据块 mid-term bank + anchor tier”； [35]
- Evict3R 的训练无关驱逐以换取更密采样。 [29]

- **中优先：事件触发预算分配（event-trigger budget allocation）**
  以 TTT3R 的“对齐置信度→闭式学习率”思想为模板，把置信度扩展为“检索深度/验证强度/恢复注入强度”的统一调度信号。 [14]

- **中优先：可选 MERG3R-style merge（小规模、触发式）**
  把 MERG3R 的“合并与置信加权 BA”用于“检索命中子图”的小规模融合，作为性能上限与可选增强，同时保持主系统仍以 feed-forward/在线为主。 [18]

- **中到低优先：Gaussian submap 实例化与全局一致渲染地图**
  若你们希望把系统推进为“可渲染持久地图”，SING3R-SLAM 明确走的是“局部 submaps → 全局 Gaussian map 融合 → 全局反馈纠正漂移”的闭环；这与“归档 bank + 重访恢复”天然契合。 [16]
  同时，Gaussian-SLAM 等也展示了“组织成 sub-maps、独立优化、无需全局常驻内存”的可扩展思想，可为“archive bank 的存储形态”提供工程参考。 [36]

最终可写进论文的最稳定位句（建议你们统一口径）
LoGeR（arXiv:2603.03269）的贡献集中在“连续流中的 chunk bridging 与尺度稳定”，而你们要证明的是“跨长时间间隔的 persistent retrieval & recovery”：把在线重建从连续传播式 hybrid memory，推进为带生命周期、可归档、可检索、可恢复的分层持久记忆建图系统。 [37]

---

[1] [3] [7] [9] [10] [12] [17] [24] [26] [27] [28] [37] LoGeR: Long-Context Geometric Reconstruction with Hybrid Memory
https://arxiv.org/html/2603.03269v1

[2] [4] VGGT-SLAM 2.0: Real-time Dense Feed-forward Scene Reconstruction
https://arxiv.org/html/2601.19887v2

[5] [34] CogniMap3D: Cognitive 3D Mapping and Rapid Retrieval
https://arxiv.org/abs/2601.08175?utm_source=chatgpt.com

[6] SLAM3R: Real-Time Dense Scene Reconstruction from Monocular RGB Videos
https://arxiv.org/html/2412.09401v3

[8] [11] [20] [25] [31] OVGGT: (1) Constant-Cost Streaming Visual Geometry Transformer
https://arxiv.org/html/2603.05959v2

[13] [23] MeMix: Writing Less, Remembering More for Streaming 3D Reconstruction
https://arxiv.org/html/2603.15330v1

[14] [22] TTT3R: 3D Reconstruction as Test-Time Training
https://arxiv.org/html/2509.26645v1

15 18 MERG3R: A Divide-and-Conquer Approach to Large-Scale Neural Visual Geometry
https://arxiv.org/abs/2603.02351?utm_source=chatgpt.com

16 SING3R-SLAM: Submap-based Indoor Monocular Gaussian SLAM with 3D Reconstruction Priors
https://arxiv.org/abs/2511.17207?utm_source=chatgpt.com

19 Streaming Visual Geometry Transformer
https://arxiv.org/html/2507.11539v2

21 35 [2603.07690] FrameVGGT: Frame Evidence Rolling Memory for streaming VGGT
https://arxiv.org/abs/2603.07690

29 [2509.17650] Evict3R: Training-Free Token Eviction for Memory-Bounded Streaming Visual Geometry Transformers
https://arxiv.org/abs/2509.17650

30 InfiniteVGGT: Visual Geometry Grounded Transformer for Endless Streams
https://arxiv.org/html/2601.02281v1

32 FrameVGGT: Frame Evidence Rolling Memory for streaming VGGT
https://arxiv.org/abs/2603.07690?utm_source=chatgpt.com

33 MeMix: Writing Less, Remembering More for Streaming 3D Reconstruction
https://arxiv.org/abs/2603.15330?utm_source=chatgpt.com

36 Gaussian-SLAM: Photo-realistic Dense SLAM with Gaussian Splatting
https://arxiv.org/abs/2312.10070?utm_source=chatgpt.com